



Centro de Investigación en Matemáticas, A.C.
Modelos Generativos Profundos

CIMAT

Project 2: Latent Score-based Generative Models

Jose Miguel Saavedra Aguilar

1 Introduction

Score-based generative models have gained significant attention in recent years due to their ability to generate high-quality samples and high likelihood from complex data distributions. These models leverage the concept of score functions, this is, the gradients of the log-density of the data distribution, to guide the generation process. By learning to estimate these score functions, score-based generative models can effectively capture the underlying structure of the data and generate new samples that closely resemble the training data.

In 2021, Vahdat et al.[4] introduced Latent Score-based Generative Models (LSGM), a novel approach that combines the strengths of score-based generative models with latent variable models. Their work motivated other researchers to explore the potential of latent-space score-based generative models, leading to further advancements in the field. One of the best-known works that followed is the one by Rombach et al.[2], who called LSGM as Latent Diffusion Models (LDM) and applied it to high-resolution image synthesis tasks with remarkable results. Later on, LDM evolved into what is now known as Stable Diffusion.

In this project, we will implement an LSGM based on the original work by Vahdat et al. but with much less resources. Their work used NVIDIA's NVAE as a latent variable model, which is a very deep and complex architecture that requires a lot of computational power to train. Instead, we will use a simpler Variational Autoencoder (VAE) as our latent variable model, and use a discrete-time score-based generative model based on Ho et al.[1] to learn the score function in the latent space, and then use it to generate new samples using a DDIM sampler [3].

While we hoped to train our model on the CIFAR-10 dataset, we were limited by computational resources and time constraints. Therefore, we opted to train our model on the simpler MNIST dataset, and then on the Smithsonian Butterflies dataset.

2 Methodology

The implementation of our Latent Score-based Generative Model (LSGM) consists of two main components: a Variational Autoencoder (VAE) and a discrete-time score-based generative model. The VAE is responsible for learning a compact latent representation of the input data, while the score-based generative model learns to estimate the score function in the latent space. Once both components are trained, we can generate new samples by first sampling from the latent space using the score-based model and then decoding the samples back to the data space using the VAE decoder.

On more explicit terms, let x be a data sample from the data distribution $p_{data}(x)$, and let z be its corresponding latent representation learned by the VAE. The LSGM framework consists of the encoder $q_\phi(z_0 | x)$, a SGM prior $p_\theta(z_0)$ and a decoder $p_\psi(x | z_0)$. The SGM prior diffuses the latent variable $z_0 \sim q_\phi(z_0 | x)$ samples into a series of noisy latent variables $z_1, z_2, \dots, z_T$ using a predefined noise schedule. This differs from the original LSGM paper, where they used a continuous-time SGM.

The training process involves three parameter sets: ϕ for the VAE encoder, ψ for the VAE decoder, and θ for the score-based generative model. The VAE is trained to maximize the evidence lower bound (ELBO) on the data likelihood as usual. The score-based generative model is trained to minimize the denoising score matching objective, which encourages the model to learn the score function of the latent variable distribution at different noise levels. On their work, Vahdat et al. proposed to use the following loss function:

$$\mathbb{E}_{t \sim \mathcal{U}(0,1)} \left[\frac{w(t)}{2} \mathbb{E}_{q_\phi(z_t, z_0 | x), \epsilon} \left[\|\epsilon - \epsilon_\theta(z_t, t)\|^2 \right] \right] \quad (1)$$

where z_t is the noisy latent variable at time t , ϵ is the added noise, $\epsilon_\theta(z_t, t)$ is the score-based model’s estimate of the noise, and $w(t)$ is a weighting function. In our discrete-time implementation, we modify the loss function to sum over discrete time steps, ending with a weighted version of the denoising score matching loss used by Ho et al.:

$$\mathbb{E}_{t \sim \mathcal{U}(1,T)} \left[\frac{w_t}{2} \mathbb{E}_{q_\phi(z_t, z_0 | x), \epsilon} \left[\|\epsilon - \epsilon_\theta(z_t, t)\|^2 \right] \right] \quad (2)$$

Where w_t is the weight for time step t , now defined on discrete time steps by the equation:

$$w_t = \frac{\beta_t}{1 - \bar{\alpha}_t} = \frac{g(t)^2}{\sigma_t^2} \quad (3)$$

where β_t and α_t are defined according to the noise schedule, and $g(t)$ and σ_t are the diffusion coefficient and noise standard deviation at time step t , respectively.

While Vahdat et al. proposed to jointly train the VAE and the score-based generative model, we opted to train them separately first due to resource constraints, and then fine-tune them together if time allowed. This approach allowed to establish a working baseline before attempting joint training.

3 Experiments and Results

We conducted experiments on two datasets: MNIST and the Smithsonian Butterflies dataset. For the MNIST dataset, we used a very simple VAE architecture with two MLP layers for both the encoder and decoder, a class conditioning embedding layer and a latent space dimension of 32. For the score-based generative model, we used the U-Net architecture provided in class with minor modifications to accept class conditioning. We trained the VAE for 100 epochs, then the score-based model for 50 epochs, and finally joint training for 20 epochs. Then, we generated samples using the DDIM sampler with stepsize 10. The generated samples are shown in Figure 1.

DDIM Latent Diffusion Samples

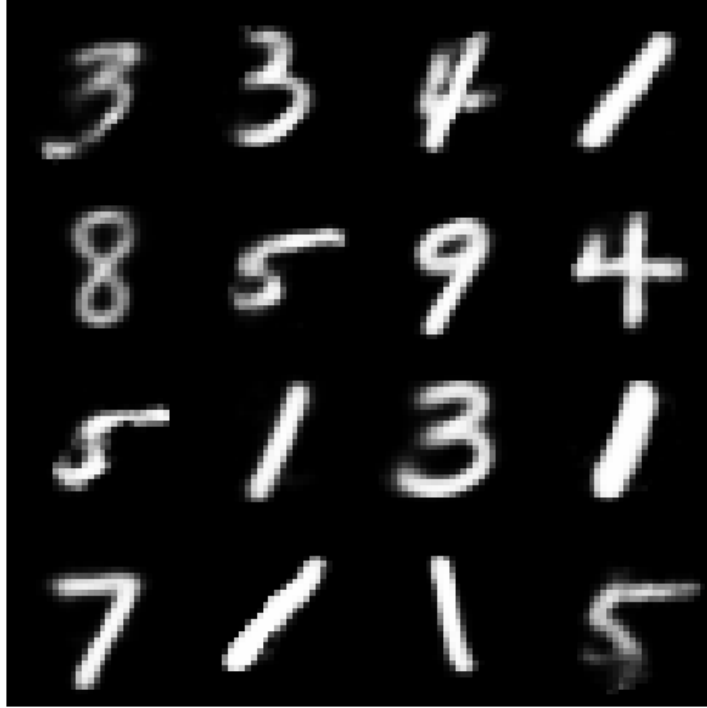


Figure 1: Samples generated using DDIM on the LSGM model trained on the MNIST dataset.

For the Smithsonian Butterflies dataset, first we tried to load a pretrained VAE model, first NVIDIA’s NVAE with pretrained weights on CIFAR-10, but the model has no exposed encoder or decoder interfaces, and even after modifying the code to extract them, the complexity of the model made it unfeasible to use with our available resources, even for the CIFAR-10 dataset. On hindsight, there are many different NVAE implementations used for different projects that may differ on architecture and training details, so maybe the one used by Vahdat et al. was more suitable for our purposes. After trying the NVAE, we also tried some other pretrained VAE models, but none gave satisfactory reconstructions on the butterfly images as they were, and we decided to build and train our own VAE from scratch for this dataset, even if it meant ending up with a model that required further training than what we could afford.

The VAE architecture used for the butterflies dataset was a convolutional, spatial aware VAE. The encoder consisted of several convolutional layers with downsampling, followed by residual blocks, and finally two convolutional layers to produce the mean and log-variance of the latent distribution. With the reparameterization trick, we sampled from the latent distribution and passed the samples through the decoder, which mirrored the encoder architecture with upsampling layers and tranverse convolutions to reconstruct the images. The latent space dimension was set to 16 channels with a spatial size of 16x16. We trained the VAE for 40 epochs using MSE reconstruction loss combined with a KL divergence term

weighted by a β factor of 0.25, making this actually a β -VAE. After training the VAE, we trained the score-based generative model using the same U-Net architecture as before without class conditioning, for 100 epochs. Finally, we trained them jointly for 10 epochs, ending with a final model that was the result of joint training. We generated samples using the DDIM sampler with 50 steps. The generated samples are shown in Figure 2.

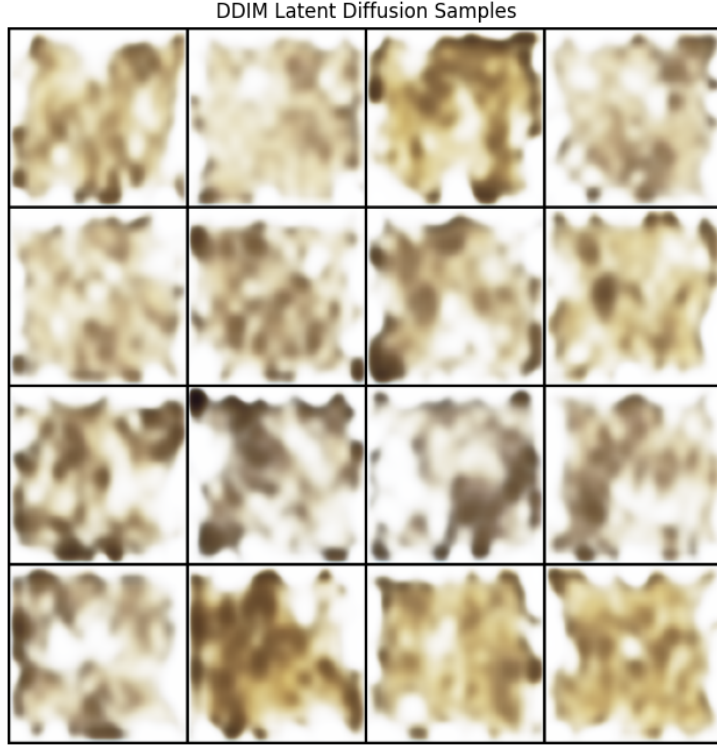


Figure 2: Generated samples from the Smithsonian Butterflies dataset using the trained LSGM.

4 Conclusions

In this project, we successfully implemented a Latent Score-based Generative Model (LSGM) using a Variational Autoencoder (VAE) and a discrete-time score-based generative model. Our experiment on the MNIST dataset validated the implementation, producing high-quality samples that closely resembled the training data. The results on the Smithsonian Butterflies dataset, while not as refined due to resource constraints, demonstrated the potential of our approach in generating complex images. This model is an example of how difficult it has become to replicate state-of-the-art generative models without access to significant computational resources, with the main contributors being part of large organizations like NVIDIA, OpenAI, or Google. However, we believe our implementation follows the principles

laid out in the original LSGM paper as closely as possible given our limitations. Despite these challenges, it is still astonishing what can be achieved by these models given the right resources.

References

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” *arXiv preprint arxiv:2006.11239*, 2020.
- [2] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” 2021.
- [3] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” 2020.
- [4] A. Vahdat, K. Kreis, and J. Kautz, “Score-based generative modeling in latent space,” in *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, Eds., vol. 34. Curran Associates, Inc., 2021, pp. 11 287–11 302. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2021/file/5dca4c6b9e244d24a30b4c45601d9720-Paper.pdf